Date 5 Maret 2026

Algoritma & Struktur Data

Problem 1 : Algorithm Analysis

For each code snippet below, perform a complete step-by-step analysis

(i). Exact operation count $T(n)$

(ii). Big-O class

(iii). Best/worst case.

Snippet A : $A = n$, $B = \frac{n(n-1)}{2}$

Line	Operation	Executions	Cost /exec
(a).	Sum = 0 ;	1	1
(b).	Outer init : i=0 ;	1	1
(b).	Outer cond : i<n ;	A+1	1
(b).	Outer incr : i++ ;	A	1
(c).	Inner init : j=0 ;	A	1
(c).	Inner cond : j<i ;	B+A	1
(c).	Inner incr : j++ ;	B	1
(d).	Body : arr[i], arr[j]	B	2
(d).	arithmetic (+,+)	B	2
(d).	assignment to sum	B	1

* $T(n) = 1 + 1 + (A+1) + A + A + (B+A) + B + 2B + 2B + B$

$= 3 + 4A + 7B$ → ganti $A = n$, $B = \frac{n(n-1)}{2}$

$= 3 + 4n + 7 . \frac{n(n-1)}{2}$

$= 3 + 4n + \frac{7}{2}n^2 - \frac{7}{2}n = \frac{7}{2}n^2 + \frac{1}{2}n + 3$

* Big-O class

$T(n) = 3{,}5n^2 + 0{,}5n + 3$

$= O(n^2)$

* Best / worst case

Best = worst. Karena struktur loop sepenuhnya ditentukan oleh n, bukan oleh isi data.

(Tidak ada if , Tidak ada break , Tidak ada kondisi yang bisa menghentikan loop lebih awal)

Snippet B : $A = \lfloor \log_2 n \rfloor + 1$

Line	Operation	Executions	Cost /exec
(a)	lo = 0	1	1
(a)	hi = n-1 (arithmetic + assign)	1	2
(a)	target = 42	1	1
(b)	found = false	1	1

Dipindai dengan CamScanner

Line	Operation	Executions	Cost/exec
(c)	while cond lo <= hi	A+1	1
(d)	Compute mid (3arith + 1assign)	A	4
(e)	arr[mid] == target	A	2
(e)	arr[mid] < target	A	2
(f)	update lo or hi (1arith + 1assign)	A	2

* $T(n) = 5 + (A+1) + 10A$

$= 11A + 6$

$= 6 + 11([\log_2 n]+1) = 17 + 11[\log_2 n]$

* Big-O class

$T(n) = 17 + 11[\log_2 n]$

$= O(\log n)$

* Best / worst case

→ Best case : target ketemu di iterasi pertama : $O(1)$

→ Worst case : target tidak ditemukan / ditemukan paling akhir : $O(\log n)$

Snippet C : $A = n$, $B = \frac{n(n-1)}{2}$, $C = \frac{n}{2}$, $D = B.C = \frac{n^2-n}{2} \cdot \frac{n}{2} = \frac{n^3-n^2}{4}$

Line	Operation	Executions	Cost/exec
(a.)	Count = 0	1	1
(b).	Outer init	1	1
(b).	Outer cond	A+1	1
(b).	Outer incr	A	1
(c).	Middle init	A	2
(c).	Middle cond	B+A	1
(c).	Middle incr	B	1
(d).	Inner init	B	1
(d).	Inner cond	D+B	1
(d).	Inner incr (k += 2)	D	2
(e).	arr[i]	D	1
(e).	arr[j]	D	1
(e).	!=	D	1
(e).	arr[k]	D	1
(e).	>	D	1
(f).	Count ++	D	1

Dipindai dengan CamScanner

Date 5 maret 2026

Algoritma & Struktur Data.

* $T(n) = 3 + 5A + 4B + 10D$

$= 3 + 5n + \cancel{4}\cdot\frac{n(n-1)}{2} + \overset{5}{\cancel{10}}\cdot\frac{n^3-n^2}{\cancel{4}_2}$

$= 3 + 5n + 2n^2 - 2n + \frac{5n^3}{2} - \frac{5n^2}{2} = 2{,}5n^3 - 0{,}5n^2 + 3n + 3$

* Big-O class

$T(n) = 2{,}5n^3 - 0{,}5n^2 + 3n + 3$

$= O(n^3)$

* Best / worst case

→ Best case : arr[i] != arr[j] selalu false (arr[k] > 0 tidak dievaluasi & count ++ tak pernah terjadi)

→ Worst case : selalu true (count ++ selalu jalan) $O(n^3)$

Problem 4 : Amortized Analysis of pushBack.

Consider the pushBack function from Problem 2, which doubles the capacity whenever size == capacity, starting from capacity = 2.

Part A – Trace

For each call, record : the value pushed, the resulting size, the resulting capacity, and the cost of that specific call (cost = 1 for normal append ; cost = 1 + old size when a resize occurs).

Call #	Value	Size after	Cap after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3 (1+oldsize = 1+2)
4	40	4	4	1
5	50	5	8	5 (resize = 1+4)
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9 (resize = 1+8)
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

Part B – Total Cost

① Sum = $1+1+3+1+5+1+1+1+9+1+1+1 = 26 = T_{12}$

② Average cost per call = $\frac{T_{12}}{12} = \frac{26}{12} = 2{,}17$

Dipindai dengan CamScanner

Date

③ Formula for Total "Copy Work" for n pushBack Operations.

→ Resizing happens when the array becomes full (size reaches the capacity) at :

2, 4, 8, 16, ...

and when the capacity grows, all the existing elements (2^t) must be copied to the new array.

The total copying work during n pushBack operations forms a geometric series :

$\boxed{2 + 4 + 8 + \ldots + 2^m,}$ where $2^m < n \leq 2^{m+1}$.

$S = 2^{m+1} - 2$ → since $2^{m+1} < 2n$, we get : $S < 2n$.

The total copy cost is : $O(n)$

Amortized Cost : The total work for n pushBack operations includes : • n normal insertions

• $O(n)$ total copying during resize

* $T(n) = O(n)$
* Amortized cost : $T(n)/n = O(1)$

Part C - Growth factor comparison

Suppose instead of doubling the capacity, pushBack grows by adding a fixed amount K=4 (new capacity = old capacity + 4)

① Redo the trace from Part A (12 calls, starting from capacity 2).

Call #	Value	Size after	cap after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3 (resize : 1+2)
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	7 (resize : 1+6)
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11 (resize = 1+10)
12	120	12	14	1

② Compare the total cost of both strategies

T_{12} (Part C) = 1+1+3+1+1+1+7+1+1+1+11+1 = 30

* Doubling (Part A) = 26
* +4 (Part C) = 30

> untuk 12 call, +4 lebih mahal (30 > 26)

Dipindai dengan CamScanner

No.

Date 5 maret 2026

Algoritma & Struktur Data

3. Derive $T(n)$ for both the doubling and the fixed-increment strategies. Show that doubling gives $O(n)$ total whereas fixed-increment gives $O(n^2)$ total. Express the amortised cost per call in each case.

* Doubling : Resize at 2, 4, 8, ... and totalcopy $< 2n$

$T(n) = n + O(n) = O(n)$

Amortized per call $= O(1)$

* Fixed increment : Resize at 2, 6, 10, 14 ... and number of resizes $\approx \frac{n}{4}$

→ Total copy-work : $2 + 6 + 10 + 14 \ldots + (4m-2)$

$S = \frac{m}{2}(2 + (4m-2)) = 2m^2$

$T(n) = n + O(n^2) = O(n^2)$

Amortized per call $= \frac{T(n)}{n} = \frac{O(n^2)}{n} = O(n)$

* Doubling → $T(n) = O(n)$

Amortized $= O(1)$

* Fixed increment → $T(n) = O(n^2)$

Amortized $= O(n)$

Doubling strategy is significantly more efficient. With doubling, the capacity grows exponentially, so the number of resizes is small.

Dipindai dengan CamScanner